TECHNICAL REPORT, THIRD EDITION

ClaimGuard
Server Deployment and Test Report

The server in full (machine, storage, services), what changed since the second report, how the final version performs, and what to do next

Runs	Full suite `server_20261101h` (about 42 minutes, 1 November 2026); follow-up runs `server_20261101i` to `l` on the final project version; first run `server_20261031` for comparison
Address	https://claimguard.kernelsi.com
Server	Ubuntu 26.04 in a Proxmox container, AMD Ryzen 7 5700X (8 cores), 16 GiB RAM, NVIDIA RTX 3080 (10 GB), one 99 GB disk
Models	Explanations: gemma3:4b. Photo reading: richardyoung/unlimited-ocr. Both served by Ollama, one at a time on the GPU.
Data	Synthetic claims only. No real patient data was used.
Branch	khalil_test, github.com/PublisherX02/Claim_Guard

Contents

1. Executive summary
2. What was deployed and how: the machine, the services, why vLLM became Ollama, and how the storage is used
3. How the tests work
4. Results of the full run, experiment by experiment (S1 to S7, S0)
5. **Follow-up: the final project version (new)**
6. Findings and recommendations
7. Problems met and how each was solved
8. Checklist before real use
9. What this test does not cover
10. Appendix: files, commands, local settings, glossary and a command for the specifications

1. Executive summary

ClaimGuard runs on the server with every feature turned on: AI explanations of claim decisions, reading of claim documents (PDF, scan and phone photos), search of the rulebook by meaning, the audit chain and the ledger. It is reachable at **https://claimguard.kernelsi.com** through Cloudflare. This third edition adds three things to the second: (1) the **full specification of the server and how its 99 GB of storage is used** (sections 2.1 and 2.6); (2) the results of the **final project version**, installed on 1 November and tested in four follow-up runs (section 5); and (3) a plain account of **why the explanation model moved from vLLM to Ollama** (section 2.5, unchanged from the second edition).

The headline is a fix and an honest correction. The API's memory problem is **closed**: after the same kind of load it used **205 MiB, where it had used 4,103 MiB** (its whole limit). The dashboard is still slow with many stored claims, and it is now shown to depend on the number of claims: 3 ms on an empty database, 1.6 s with 15,000 to 18,000. The search speed of the first report (2 to 4 ms) came from the answer cache; a question asked for the first time takes about 1 second alone and about 8 seconds on the server when eight are asked at once.

205 MiB
API memory after load (was 4,103 MiB, at its limit)

0 errors
10 minutes at 100 claims a minute

0 lost
claims lost in 5 restarts (run h)

90.0 %
document fields read exactly

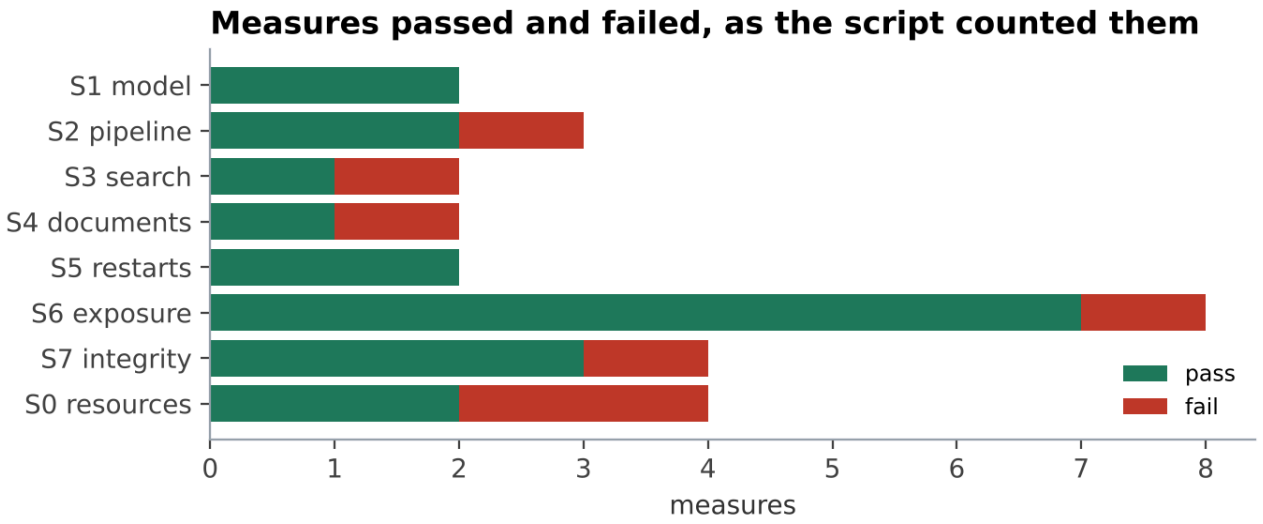


Figure 1. Measures passed and failed per experiment in the full run h, as the script counted them (20 of 27 passed; S7 shows the repeated check). Section 5 reports the follow-up runs.

Where things stand

Finding	Status	Importance
API memory at its 4 GiB limit; HTTP 502 for a moment	**Closed. Final version: 205 MiB after load (section 5.3)**	**Done**
Dashboard and health panel about 1.6 to 1.8 s with 15,000 to 18,000 stored claims	**Open. Cause confirmed: it grows with stored claims (3 ms when empty). Needs cached summaries for large volumes (section 5.5)**	**High at volume**
Phone photos read correctly in 60 % of fields (target 95 %)	**Open (not re-tested; `OCR_DPI` 200 untried)**	**High for document intake**
Search by meaning: 23 of 26 right (88.5 %, line 90 %); 4 of 5 off-topic refused	**Open, one question short of the line; one of the three misses was a timeout (section 5.6)**	**Medium**
Health probe "flow" degraded	**Explained: it is degraded while test claims wait with nobody on shift; "ok" on the emptied database (section 5.7)**	**Low**
Health probe "timestamp" unknown	**Open: no external timestamp service configured**	**Low**
Explanations: 210 a minute, 5 % above target (376 with vLLM)	**Open, a cost of the change to Ollama (not re-run)**	**Medium**
Ollama container used 99 % of its 5 GiB limit (run h)	**Open, not re-checked**	**Medium**
Disk: one 99 GB volume, 75 % used, filled up twice	**Open. A split and routine are proposed in section 2.6**	**Medium**

What improved since the first report

- **Documents.** Image-only PDFs and scans are read at 100 % of fields (97.3 % and 98.2 % before). Photos rose from 50 % to 60 %. Overall 90.0 % of fields (was 86.4 %).
- **One card, both models.** The explanation model and the vision model share the 10 GB card without manual switching. GPU memory peaked at 5.8 GiB (58 %).

- **Memory and steady load.** The final version streams claims instead of loading them all; the API stayed near 200 MiB under load, and a 10-minute steady load at 100 claims a minute ran without one error.
- **Safety of data and integrity.** Five services were restarted under load with 0 claims lost (run h). The audit chain and the ledger verify; the security log has 86 signed checkpoints.

2. What was deployed and how

2.1 The machine

The server is a Proxmox container named `khalil`. Everything below was read from the machine during the runs; items marked "not recorded" were not measured and are listed in the command of section 10.5, so they can be added.

Item	Value
Type	Proxmox LXC container (its disk is named `pve-vm--138--disk--0`, so it is container 138)
Operating system	Ubuntu 26.04 LTS, kernel 7.0.14-17-pve, glibc 2.43
CPU	AMD Ryzen 7 5700X, 8 cores visible to the container
Memory	16 GiB (17,179,869,184 bytes), raised from 12 GiB after a build froze the server. With all 8 containers running and idle: about 4.8 GiB used, 3.0 GiB file cache, 11 GiB available. During run h the least available was 3.4 GiB; the busiest moment used 81 %
Swap	Not recorded (it was full during the build freeze)
GPU	NVIDIA GeForce RTX 3080, 10,240 MiB, driver 615.71.09. Peaks seen: 5,909 MiB memory, 100 % utilisation, 69 °C, 329 W
Disk	One volume of 99 GB mounted as the root filesystem (`/dev/mapper/pve-vm--138--disk--0`); 71 GB used, 24 GB free (75 %) at 19:07 on 1 November. Section 2.6 shows how it is used
Network	Visitors reach it through Cloudflare. Only ports 80 and 443 listen on the server (Caddy); no other service is published
Software	Docker 29.9.2, Docker Compose 5.6.1, Python 3.14.4 (for the test scripts), Caddy 2, MongoDB 7, Redis 7, Ollama (latest image)

> **Memory limits add up to more than the machine has.** The containers' limits are api 4 GiB, worker 3 GiB, mongo 2 GiB, embed 4 GiB, reader 2 GiB, ollama 5 GiB, redis 320 MiB and proxy 128 MiB: 20.4 GiB in total on a 16 GiB machine. This is normal (limits are ceilings, not reservations), and the measured peaks of run h added up to about 13.9 GiB, but it means that if everything peaked at once the machine would run out first. The API's peak has since dropped (section 5.3), which gives more room.

2.2 The services

ClaimGuard runs as eight containers managed by Docker Compose. Each does one job:

Container	Role	Notes
proxy (Caddy)	Receives all web traffic, handles HTTPS	The only service that listens on the network (ports 80 and 443)
api	The ClaimGuard API and the web console	Written in Rust, memory limit 4 GiB (uses about 200 MiB since the final version)
worker	Processes the queue of claims	Rust, memory limit 3 GiB
mongo	Database	MongoDB 7
redis	Queue and short-term storage	Redis 7
ollama	Runs both AI models on the GPU, one at a time	gemma3:4b (explanations) and richardyoung/unlimited-ocr (photos); replaces the vllm container
embed	Search by meaning	bge-m3 model, runs on the CPU
reader	Reads documents	Tesseract plus the vision model; the two readings are cross-checked

2.3 How a visitor reaches it

A browser opens `claimguard.kernelsi.com`. Cloudflare answers first, then forwards the request to the server, where Caddy passes it to the API. A Cloudflare rule sends anyone who types `http://` to `https://`. Inside the server, no other service is exposed.

2.4 Settings that must survive every update

Four files hold the changes that make this server work. A project update must not overwrite them. The update to the new project version followed this rule: a backup was taken first, and the new files were copied in with these excluded.

File	Why it matters
`.env`	Passwords, keys, model settings, the site name
`deploy/Caddyfile.internal`	Sends the real domain name to the API (`header_up Host`) and sets a default certificate name; without it the login fails with "That request came from another site"
`deploy/rust/compose.gpu.yml`	Gives the GPU containers access to the card inside the container, mounts extra GPU libraries, sets memory limits
`deploy/rust/compose.local.yml`	**New.** This server's own settings: the Ollama service, the address of the explanation model (Ollama instead of vLLM), the vision model for the reader, the 4 GiB API limit. The project never overwrites it and keeps it out of Git (document 47, "Keeping local changes across updates"). Its content is in section 10.4

2.5 Why the explanation model moved from vLLM to Ollama

The short answer. The vision model that reads phone photos needs several gigabytes of GPU memory while it works, and vLLM keeps most of the card for itself. Both could not run on the 10 GB card at the same time. Serving both models through Ollama, which loads one at a time and swaps them automatically, solved this. It is also how the project was tested on the developer's 8 GB laptop.

What was observed.

- With Tesseract alone, phone photos were read correctly in only 50 % of fields (first report). The project documentation shows a vision model lifting photos to 88.1 % on the build PC, so the vision

model was added.

- vLLM was configured to reserve 85 % of the card (7.7 GiB measured). The vision model needs about 2.8 GB for its weights and a working buffer of about 2.4 GB for a large photo. Only about 1.4 GiB was left. The vision model ran out of GPU memory and crashed: all 80 photos in a direct test returned HTTP 500, and the reader silently fell back to Tesseract alone, so the photo score did not change (50 %).
- Each crash also wrote a core dump inside the Ollama container. About 80 crashes filled 30 GB and, together with the earlier build cache, filled the disk twice.
- With vLLM stopped, the same vision model read photos in about 2 seconds each and the photo score rose to 60 %. So the model works; the card was the limit.

The options that were weighed.

Option	Result
Keep vLLM and add the vision model beside it	**Out of GPU memory (HTTP 500 on every photo). Not usable on a 10 GB card; document 47 also says the project's vision profile does not fit beside Gemma on 10 GB**
Give vLLM less of the card	**Not enough. A first try at 55 % had already failed with "no memory for the KV cache"; and the vision model alone used about 5.6 GiB of the card when loaded, so the two together exceed 10 GiB. (A value of 70 % was considered and not tried, because this arithmetic does not close.)**
Stop vLLM by hand whenever photos are imported	**Works (photos at 60 %, 2 s each) but needs a person to switch the models by hand every time**
Run the vision model on the CPU	**2 to 4 minutes per photo; 39 of 40 documents failed in that test because the test client gave up after 60 seconds**
Serve both models through Ollama (chosen)	**One model on the card at a time, swapped automatically. Gemma passed the rulebook check (15 of 15) and the photo test ran after it, then Gemma passed again**

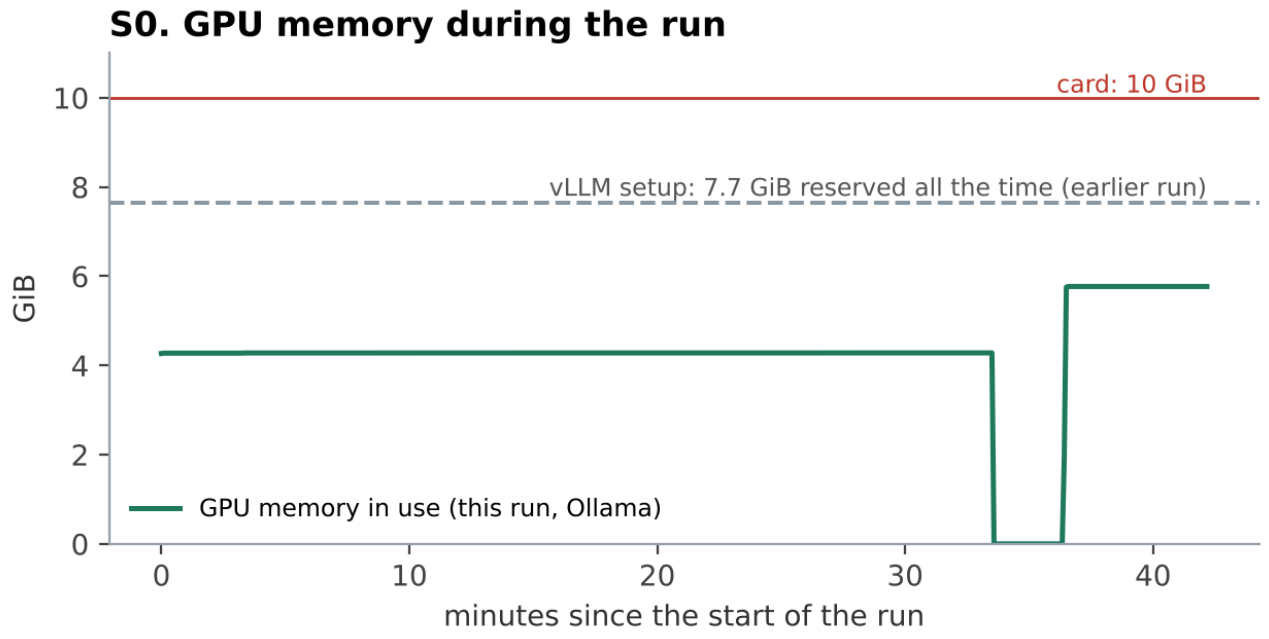


Figure 2. GPU memory in use during this run (green) against the vLLM setup (dashed grey, 7.7 GiB reserved at all times) and the 10 GiB card. The brief dip to zero is a moment with no model loaded; the step to 5.8 GiB is the vision model being loaded for the photo test.

What it costs.

	vLLM setup (first run)	Ollama setup (this run)
Explanation model	google/gemma-3-4b-it, fp8 weights	gemma3:4b, Ollama's 4-bit build (3.4 GB)
GPU memory	7.7 GiB reserved at all times	4.3 GiB with Gemma; peak 5.8 GiB with the vision model
Best explanations per minute	**376**	**210 (target 200)**
One request at a time	0.62 s	0.47 s
Requests that run at once	4 or more	2 (a setting; more needs more memory)
Phone photos, fields read exactly	**50 % (vision model not usable)**	**60 %**
Rulebook check with the model	15 of 15	15 of 15
Switching between models	Not possible	Automatic; the first photo after a long pause took about 70 s once (model loaded from disk); the photo test with swaps in between had a slowest photo of 8.8 s

> **Risks to keep in mind.** (1) Explanations are about 44 % slower under load; the margin over the 200 a minute target is small. (2) The 4-bit build is not the same file as the fp8 weights; the rule check passes, but the quality of the written explanations was not scored in either setup. (3) The vision model comes from a third-party conversion; the project's document 45 advises converting the official weights yourself and recording hashes before production. (4) The Ollama image is not pinned to a version yet. **When to go back to vLLM:** a card with 16 GB or more, or an explanation load above about 200 a minute.

2.6 How the 99 GB of storage is used

The layout. The server has **one volume** of 99 GB, shown as the root filesystem. As far as the outputs seen show (`df` lists only this volume), nothing is split into separate disks: the operating system, Docker's images and build cache, the databases, the AI models, the project files, the backups and the test results all share the same 99 GB. That is why a few large things (a build cache, then core dumps) filled the whole disk twice on 1 November, and why MongoDB crashed when it happened.

What lives on the volume	Where	Size seen
Operating system and tools	`/`	Not recorded
Docker images (claimguard-rs, reader, Ollama, Mongo, Redis, Caddy, embed)	Docker storage	Not recorded as a total
Docker build cache	Docker storage	**About 20 GB after one rebuild; 5.06 GB freed after the final update's build**
AI models in the Ollama volume (`ollama_data`)	Docker volume	gemma3:4b 3.4 GB; the vision model's weights about 2.8 GB (project notes); together about 6 GB
Search model (bge-m3, for the embed container)	Model folder	Not recorded
MongoDB data (claims, events, accounts)	Docker volume	Test data of 18,325 claims and related records: a compressed dump of **348 MB**
Project folder `~/Claim_Guard-main` with `outputs/`	Home folder	Not recorded; the results of runs i to l are 38 KB
Backups: `~/backup-before-update*`, `~/mongo-test-data.gz`, zips	Home folder	The Mongo dump is 348 MB; the others are not recorded

What happened to the disk on 1 November

Event	What filled it	What fixed it
Disk 100 % full, first time	About 20 GB of Docker build cache from rebuilding the image	`docker builder prune -af`; unused images removed; disk fell to 82 %
Disk 100 % full, second time	Core dumps written by the crashing vision model: the Ollama container's own layer grew to 30.1 GB	Container removed (not its volume) and `ulimits core 0` added; 25 GB free afterwards
Final update build	Another rebuild; disk at 80 %	`docker builder prune -af` freed 5.06 GB
After the follow-up runs	Results, a 348 MB database dump	None needed: 71 GB used, 24 GB free (75 %)

> **The Proxmox host is also tight.** It was 89 % full earlier on 1 November. The container's 99 GB is carved out of the host's storage, so a full host would stop this server however much space the container thinks it has. Check the host first before enlarging the container's disk.

A proposed split

This is a **proposal**, not what exists today, and its sizes are budgets to adjust once the numbers of section 10.5 are collected. The idea is to stop one runaway item from filling everything, and to keep backups off the machine they protect.

Part	Budget	How	Why
System and Docker engine (images, build cache)	35 GB	The root volume	Images are rebuilt; the build cache is disposable. Prune it after every build
Models (Ollama volume, search model)	15 GB	A separate virtual disk (Proxmox mount point) for `ollama_data`	Holds about 6 GB today; room for two more models. A crash dump then cannot eat the database's room
MongoDB data	25 GB	A separate virtual disk for the Mongo volume	Grows with real claims; a full disk crashed it once
Backups	Off this machine	Another Proxmox storage or a different machine	A backup on the same disk is lost with the disk; the backup folder also holds the encryption key
Free headroom	At least 20 %	Alert at 80 % used	A rebuild needs about 20 GB of temporary space

Routine. After each build: `docker builder prune -af`. Before a long test: `df -h /`. Weekly: `docker system df`. Keep `ulimits: core 0` on the model container. Never remove a volume (`-v`) when recreating a container.

3. How the tests work

The suite lives in `experiments/server/`. Section 4 reports the full run, made as `run_all.py --run server_20261101h` with the same targets as before (document 65 of the project). The 30-minute steady load, the restart test and all other experiments ran again in full. After the final project version was installed, the pipeline (S2), integrity (S7) and search (S3) experiments were repeated in shorter runs (section 5).

Code	Question	Method
S1	How fast is the AI model?	Requests for explanations, 1 to 64 at the same time
S2	How fast is the whole claim pipeline?	Batches of 10 to 500 claims, then 30 minutes of steady load
S3	Does search by meaning find the right rule?	26 real questions and 5 off-topic strings
S4	Are documents read correctly?	40 test documents: digital PDF, image-only PDF, scan, photo
S5	Is anything lost when a service restarts?	Restart five services while claims arrive and count them
S6	What does the internet see?	Port scan, encryption test, headers, anonymous requests
S7	Is the audit trail intact?	Health panel, audit chain and ledger verification
S0	Are resources enough?	Memory, CPU and GPU sampled every five seconds during the run (507 samples, 42 minutes)

Notes on this run. (1) S1 was pointed at Ollama directly, because the script normally looks for the vLLM container. (2) The run started with about 8,500 test claims already stored, about five times as many as the first run started with (about 1,700), which matters for S2 and S0. (3) S3's speed test was blocked by the search rate limit (295 requests answered with HTTP 429), because the temporary high limit of the first run had been removed; the quality result is not affected. (4) The first attempt of S7 received HTTP 502 from the API; the check was repeated by itself a few minutes later (section 4). Only measured results are reported below.

4. Results of the full run (server_20261101h), experiment by experiment

> This section reports run h as it was measured, on the project version installed before the final one. Where the follow-up runs of section 5 changed the picture, a line says so.

S1. The AI model on the GPU: pass, but slower than with vLLM

Each request asks the model for one explanation (about 250 tokens in, up to 160 out). With one request at a time the model answers in 0.47 seconds, faster than vLLM did (0.62 s). With more requests together, throughput rises to about 202 a minute at 2 requests and then stays flat at 198 to 210 a minute: Ollama runs 2 requests at once, and the rest wait in line. The best value, **209.9 a minute** at 16 requests, is 5 % above the target of 200. No request failed (560 requests).

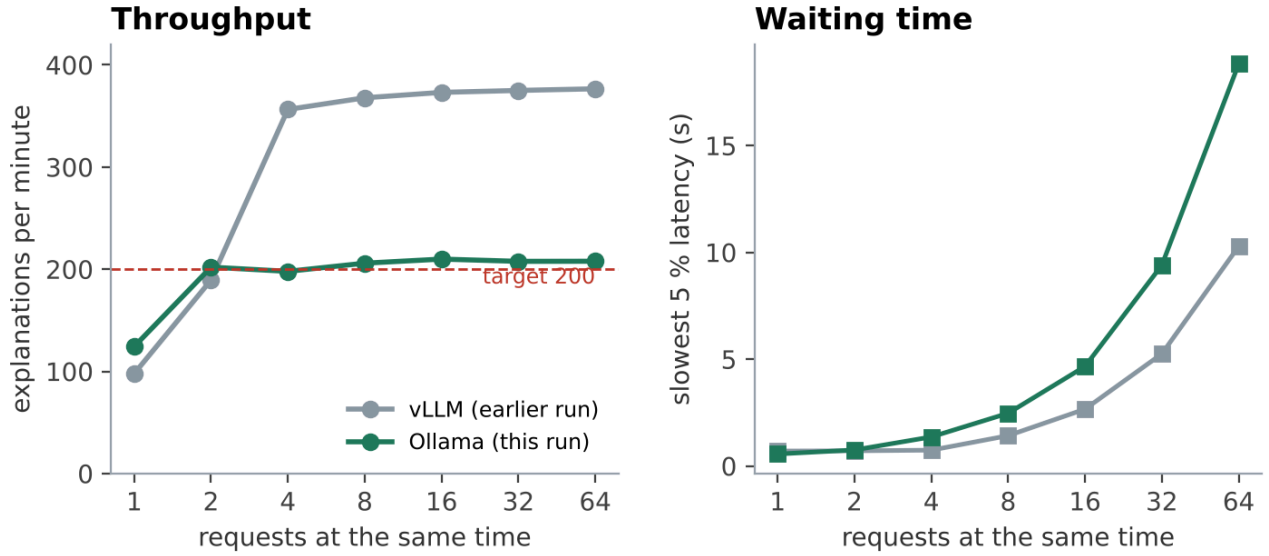


Figure 3. Explanations per minute (left) and waiting time of the slowest 5 % (right), with vLLM (grey, first run) and Ollama (green, this run).

At the same time	Requests	Errors	Per minute	Median (s)	Slowest 5 % (s)	Tokens/s
1	16	0	124.3	0.47	0.56	148.6
2	16	0	202.0	0.58	0.75	243.6
4	16	0	197.9	1.19	1.36	238.8
8	32	0	205.9	2.32	2.47	248.6
16	64	0	209.9	4.56	4.66	253.2
32	128	0	207.7	9.20	9.40	248.0
64	256	0	207.8	18.49	18.82	250.9

> **What it means.** The target is met, with little room. A reviewer waiting for one explanation gets it in about half a second; under heavy load the wait grows quickly (4.6 s at 16 requests). The GPU memory peak during this test was 4,379 MiB, so there is memory left to run more requests at once if needed (the setting is `OLLAMA_NUM_PARALLEL`), at the price of more memory.

S2. The claim pipeline: throughput passes, read pages are slow

Claims travel through proxy, API, queue, engine and store. The best rate was **4,140 claims a minute** with batches of 500, about 20 times the target and below the 11,721 of the first run. During the whole test the GPU was idle (0 % utilisation in the 30-minute steady load), so the explanation model is not what limits this part. The most likely cause is the amount of data already stored: about 8,500 claims at the start (about 1,700 in the first run), with the API already using 3.6 of its 4 GiB. This was not proven then; section 5.5 shows it is the amount of stored data.

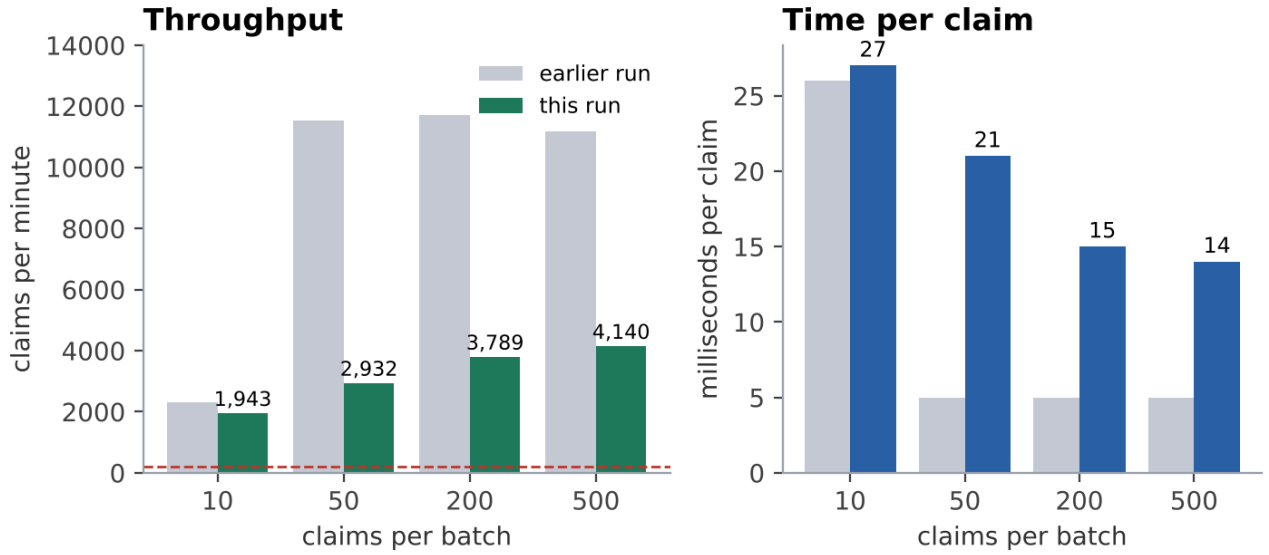


Figure 4. Claims per minute and milliseconds per claim by batch size: first run (grey) and this run (colour). Dashed line: target of 200 a minute.

Batch	Batches	Claims	Errors	Per minute	Batch done (s)	ms per claim
10	6	60	0	1,943	0.27	27
50	6	300	0	2,932	1.04	21
200	4	800	0	3,789	3.06	15
500	2	1,000	0	4,140	7.24	14

Reading the system, idle and right after a heavy load

This is the clearest change from the first run. In the first run the dashboard and the health panel answered in a few milliseconds when the system was idle. Now they take **about 2 seconds even when idle** (1.9 s and 2.0 s) and 2.6 to 2.7 s for the slowest 5 % right after the load, above the 2-second limit. The claims list stays fast (2 ms). The pages that count and summarise all stored claims are the slow ones, which points to the data volume and to the API running at its memory limit. The follow-up confirmed it (section 5.5).

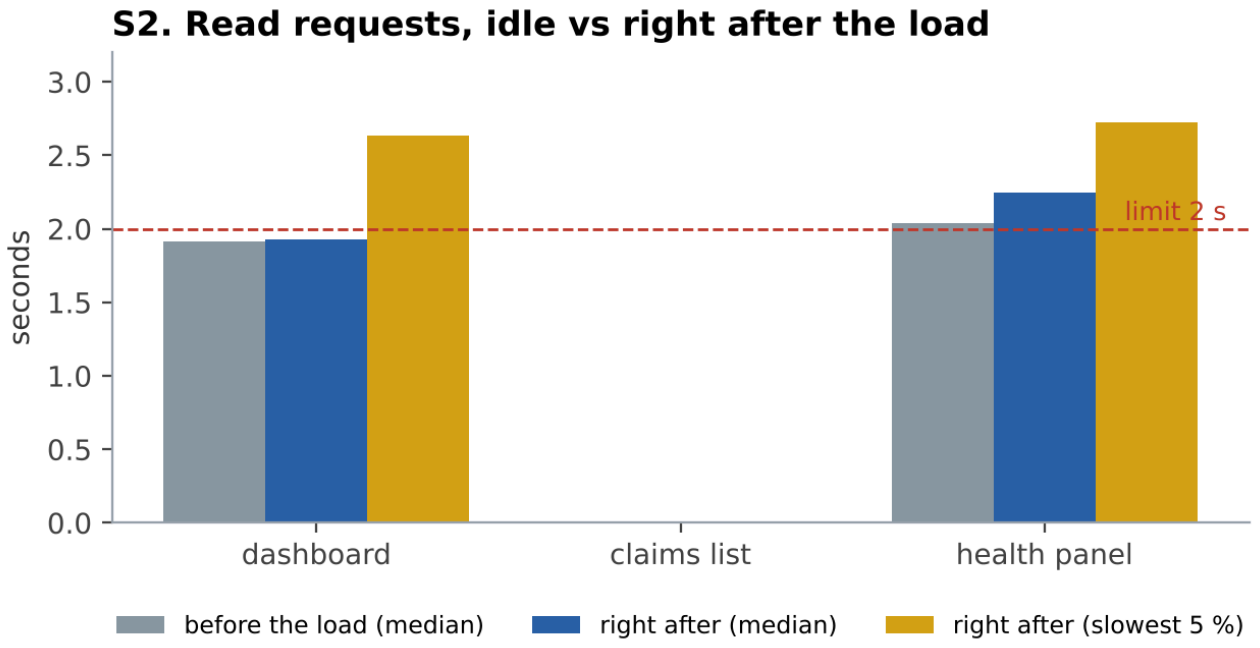


Figure 5. Answer time of three read requests, idle and right after the load. Limit: 2 seconds.

Steady load for 30 minutes: pass

3,000 claims were sent at 100 a minute for 30 minutes. **No batch failed.** The typical batch took 0.26 seconds throughout; the slowest 5 % reached 2.3 s once, at minute 12. The one failed batch of the first run did not repeat. This run recorded the error text of any failure, and there was none, so the earlier failure's cause remains unknown.

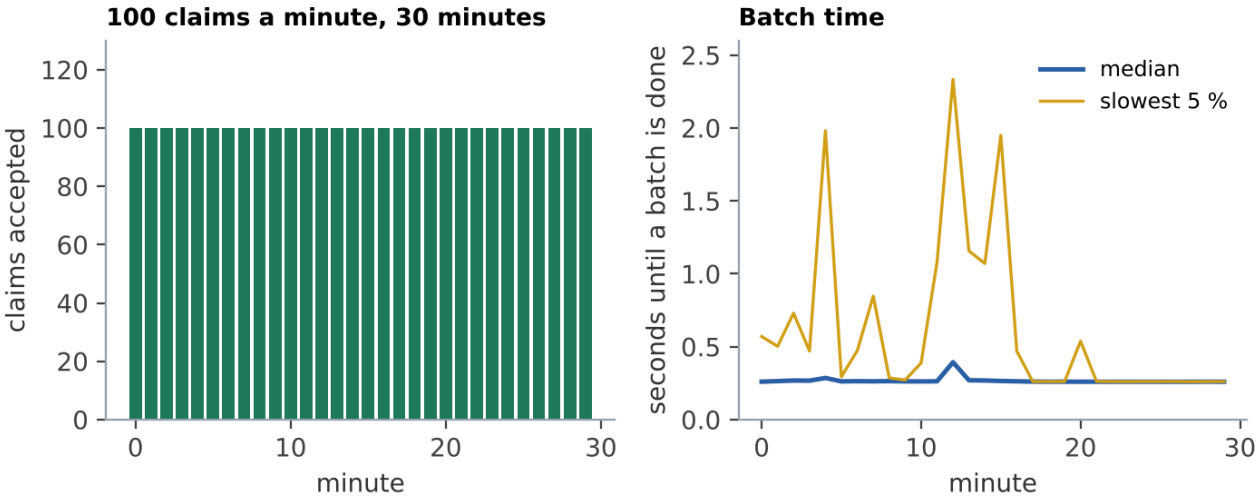


Figure 6. Claims accepted per minute (left) and time until a batch was done (right) during the 30-minute steady load.

S3. Search by meaning: quality passes, one off-topic string accepted (unchanged)

Staff type a question in their own words and the system finds the matching rule. **24 of 26 questions returned the right rule (92 %, target 90 %)**, the same as before. The same two questions missed: "claim line without a service date" (rule R001) and "how many days after the service can a claim be submitted" (rule R014 and two policy tables). **4 of 5 off-topic strings were refused**; one was answered, as before. The refusal level is a setting (`EMBEDDER_THRESHOLD=0.485`); raising it refuses more unrelated text but may also refuse genuine questions, and the pass line is only 2 questions away.

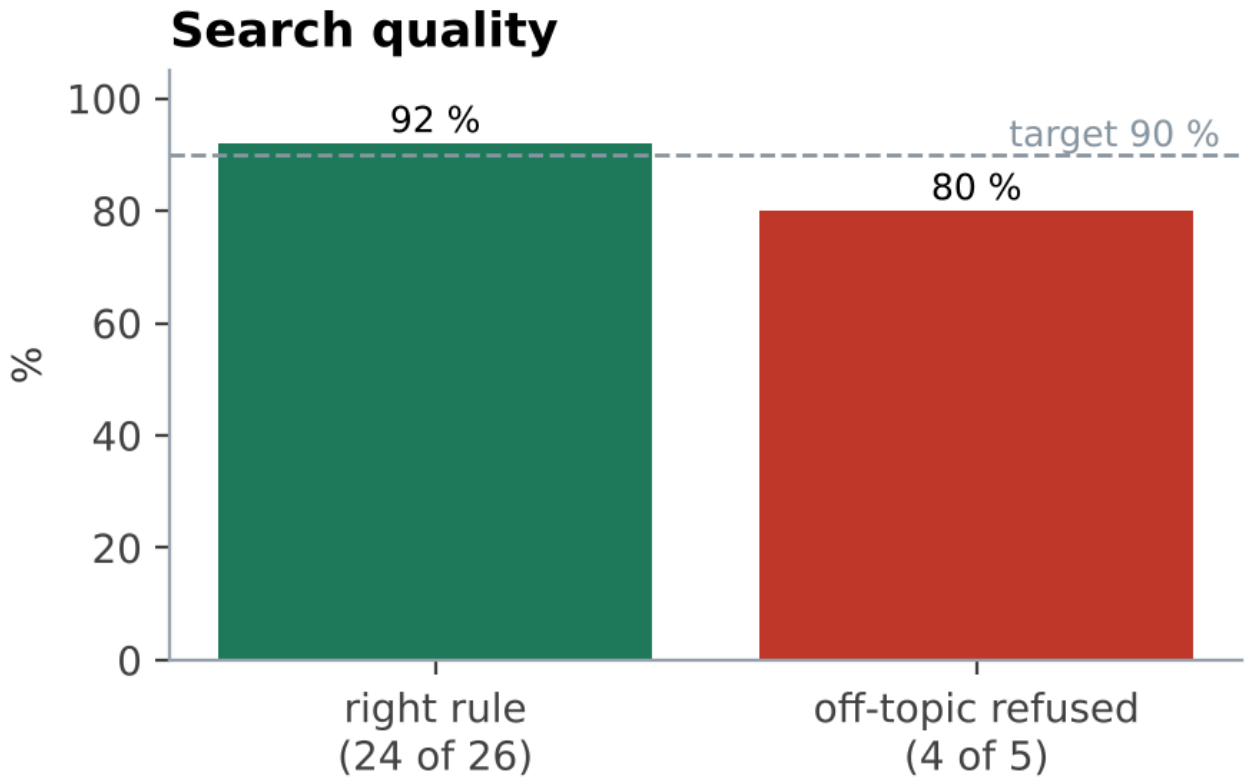


Figure 7. Share of questions answered with the right rule and share of off-topic strings refused.

Speed was not measured in this run. Of the speed test's requests, 295 were refused with HTTP 429 because the search rate limit was back at its normal value; only one level (1 at a time) produced answers. The first run's speed (a median of 2 to 4 ms, up to 1,819 answers a second at 8 at once) was measured with a temporary high limit and stands as the reference only for repeated questions; section 5.6 shows that the speed of that first run came from the answer cache and gives the honest speed. The 26 quality questions took a median of 3.8 s each in this run (slowest 5 %: 5.2 s); the first report did not record this figure, so there is nothing to compare it to, and what the time consists of was not checked.

S4. Documents to claims: much better, photos still below target

40 documents were read, 0 failed to read. The reader now runs Tesseract and the vision model together and compares the two readings. The test with the vision model on the GPU was run three times with the same result (the last time with the explanation model loaded on the same card), so the figures are stable.

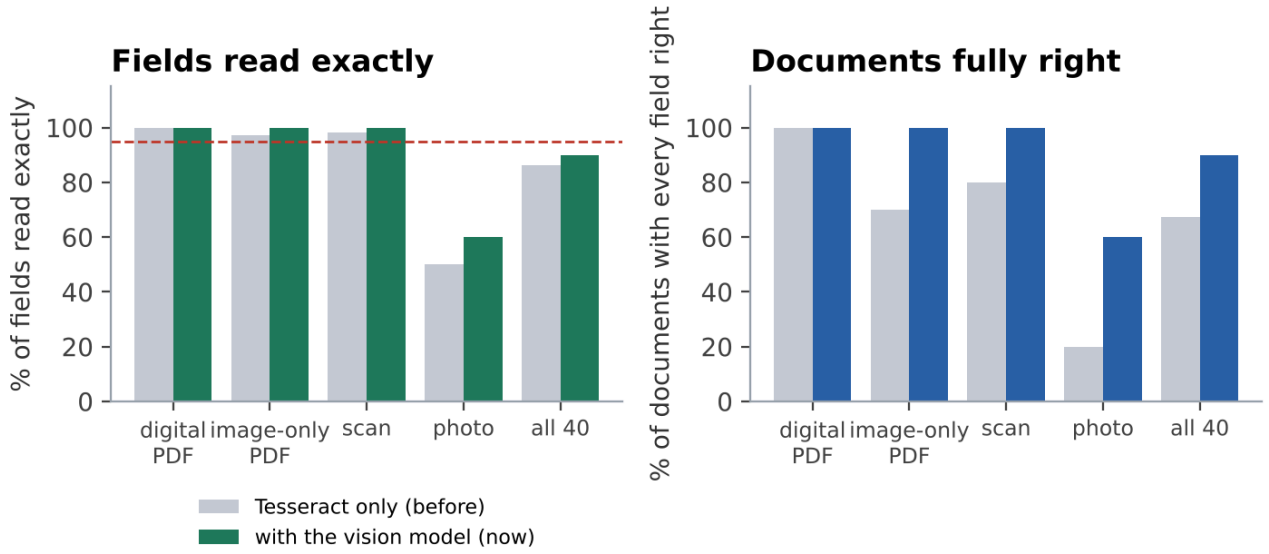


Figure 8. Left: share of fields read exactly, Tesseract only (grey) and with the vision model (green); red line: 95 % target. Right: share of documents with every field right.

Document type	Documents	Fields exact (before)	Fields exact (now)	Fully right (now)	Median time (s)
Digital PDF	10	100.0 %	100.0 %	100 %	0.0
Image-only PDF	10	97.3 %	100.0 %	100 %	3.1
Scan	10	98.2 %	100.0 %	100 %	3.2
Photo	10	50.0 %	**60.0 %**	**60 %**	2.3
All	40	86.4 %	**90.0 %**	**90 %** (was 67.5 %)	

What it means. PDFs and scans now meet the 95 % target in all three types. Phone photos improved but remain at 60 %, so the overall 90.0 % is still below the 95 % target. The project's own measurement of 88.1 % for photos came from its evaluation set on the build PC and is not directly comparable. For now, ask staff to submit PDFs or scans where possible, and keep a human check on every document read. A next step is to test a higher image resolution for the vision model (the server uses 130 dots per inch, lowered when memory was short; the project default is 200), since the card now has the room.

S5. Restarts under load: pass

Each service was restarted while a background client kept sending claims in batches of five. After each restart, the queue total was compared with the number of claims the system had acknowledged. A claim that was acknowledged and then missing would be the failure that matters.

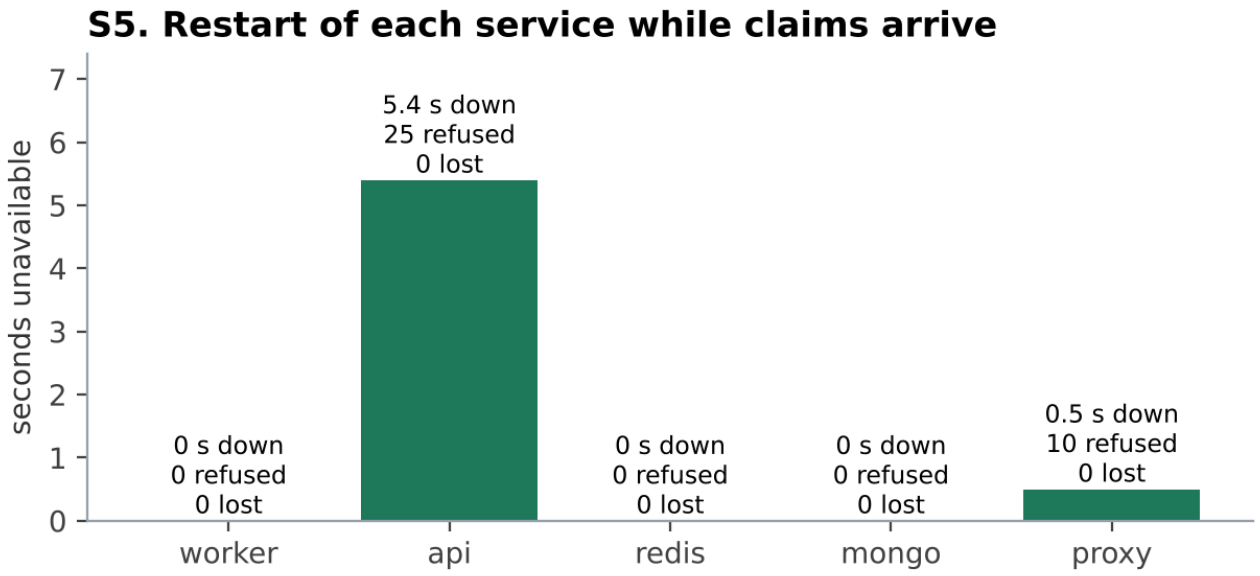


Figure 9. Seconds the service was unavailable, requests refused and claims lost, per restarted service.

Service	Restart (s)	Unavailable (s)	Back	Sent	Acknowledged	Refused	Lost
worker	2.7	0.0	yes	375	375	0	**0**
api	5.7	5.4	yes	280	255	25	**0**
redis	0.4	0.0	yes	335	335	0	**0**
mongo	0.5	0.0	yes	315	315	0	**0**
proxy	0.5	0.5	yes	185	175	10	**0**

What it means. The same result as before. Nothing acknowledged was lost and every service came back. Restarting the API makes the site unavailable for about 5 seconds, the proxy for half a second; requests sent in that window are refused, not silently dropped, so a client that retries loses nothing. Plan updates for a quiet moment.

S6. Exposure seen from the network: pass in practice

The scan looks at the server like an attacker would, from outside, and ran against the domain, so Cloudflare sits between the test and the server. The result is the same as in the first report: 7 of 8 checks pass.

Check	Result
HTTPS open	**Yes**
TLS 1.0 and 1.1	**Refused**
HTTP on port 80	**Redirects to HTTPS**
Security headers (HSTS, content-type protection, content security policy, framing protection, no server version)	**All present**
Anonymous request to the API and to the health panel	**Refused (401)**
`/healthz` shows only a minimal answer	**Yes**
Internal ports (database, Redis, model server, embedding, reader, Ollama, app, SSH) closed	**Reported as not all closed: Cloudflare answers on port 8080 (test artefact)**

On the server itself, `ss -tlnp` shows only Caddy listening, on ports 80 and 443. Nothing listens on 8080, so the reading is Cloudflare's own port. Ollama's port 11434 is not published to the network either: it is reachable only from the other containers.

S7. Integrity and audit: pass, with one finding and one optional probe unknown

The first attempt, at the end of the document test, received **HTTP 502 from the API** on all four requests, so it tested nothing. The API was at its memory limit at that moment (4,103 of 4,096 MiB in the samples) and may have been restarting; this was not confirmed, but it is a real signal that the API was unavailable for a short time. The check was run again by itself a few minutes later, once the API had settled:

Check	Result
Health panel answers	**Yes (HTTP 200, 2.2 s)**
Audit chain verifies	**Yes: 19,197 events, key configured, anchor signed**
Ledger verifies	**Yes: 77 checkpoints over 19,199 log rows, signing key present**
Health probes	**Overall "degraded": every probe is "ok" except `timestamp` unknown (no external timestamp service, as in the first run) and `flow` degraded**

The `signals` probe, which was degraded in the first run because of failed test sign-ins, is "ok" now. The `flow` probe is new in this project version; section 5.7 explains what makes it degraded.

S0. Resources during the run: warnings on two memory limits

Memory, CPU and GPU were sampled throughout the 42 minutes (507 samples). The server never had less than 3.4 GiB of its 16 GiB available; the busiest moment used 81 % of it. The GPU peaked at 5,909 MiB of 10,240 MiB (58 %), 100 % utilisation, 69 °C and 329 W, during the model test and the photo test.

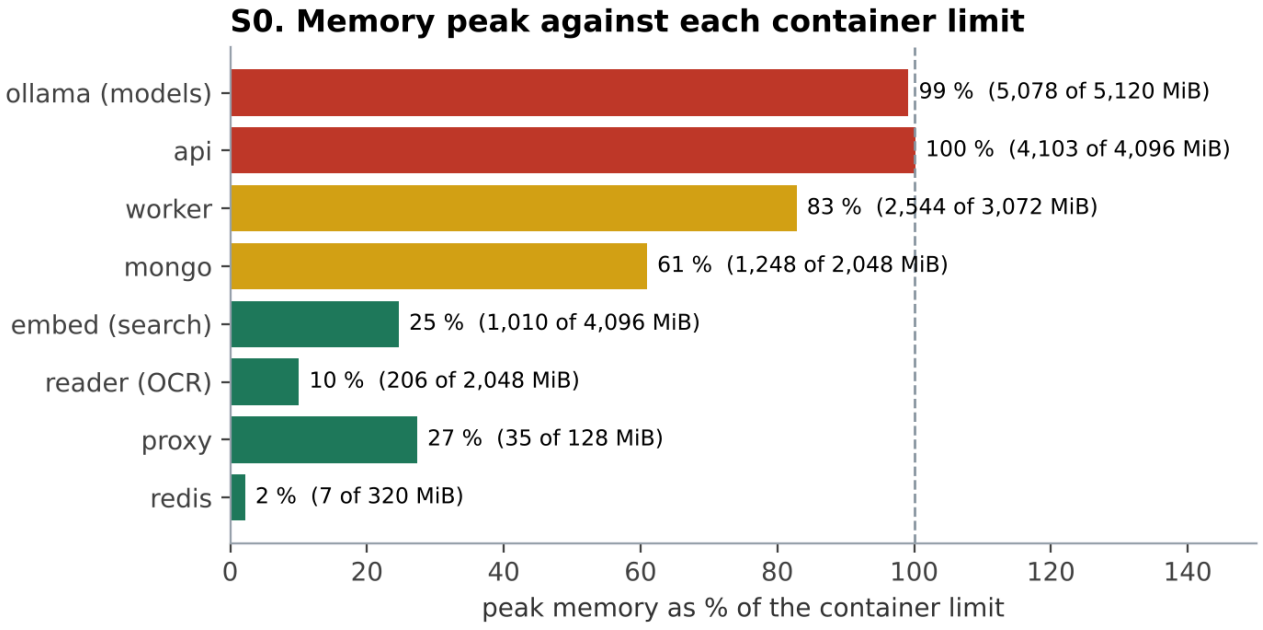


Figure 10. Peak memory of each container as a share of its limit.

Container	Peak (MiB)	Limit (MiB)	Share
ollama	5,078	5,120	**99 %**
api	4,103	4,096	**100 %**
worker	2,544	3,072	83 %
mongo	1,248	2,048	61 %
embed	1,010	4,096	25 %
reader	206	2,048	10 %
proxy	35	128	27 %
redis	7	320	2 %

The API is still the one to watch, and Ollama joins it. The API's limit was raised from 3 to 4 GiB after the first run, and it still touched the limit with about 15,000 test claims stored, because it keeps claims in memory. Its memory also grew by about 1.1 GiB an hour during the pipeline test while claims were added. The final version removed this growth (section 5.3). Ollama's 99 % may be mostly file cache from reading the model files (not checked); if its container is ever killed (exit code 137), raise its limit.

5. Follow-up: the final project version (runs i to l)

5.1 What was installed

The final project version was installed on 1 November, with the four local files kept (`.env`, the proxy file, `compose.gpu.yml` and `compose.local.yml`), after a backup. The new image built in 341 seconds with no problem, all 8 containers were healthy, the rulebook check passed 15 of 15 with gemma3:4b, and the console answered with HTTP 200. The API used **40 MiB right after the restart**, with the same 15,000-plus stored claims in the database.

What the new version changes, from the project's own documents 66 to 68 (not measured on this server unless stated): claims are now read in a stream, so the API's memory no longer follows the number of claims; a request with an empty `Transfer-Encoding` header is no longer taken for one with a body; the "Ask the rulebook" screen shows "Searching…"; signing out works without an error; a viewer account no longer sees the Brand page; operators can have working-hour schedules and difficulty bands; after a password change the person signs in again. The project reports that its question search, with a local model, answered 47 of 50 test questions correctly on the developer's machine (document 68). That figure was **not repeated on this server**; section 5.6 reports what this server measured.

5.2 The runs

Run	What	Database at the start
`server_20261101h`	The full suite, S0 to S7 (section 4)	About 8,500 test claims
`server_20261101i`	S2 (pipeline, 10-minute steady load) and S7, on the final version	About 15,000 claims
`server_20261101j`	S2 (2-minute steady load) and S7, after the test claims were removed	Empty (accounts kept)
`server_20261101k`	S3 search: no valid result. 282 requests were refused with HTTP 429 by the search rate limit	Empty
`server_20261101l`	S3 search, with the rate limit raised for the run and set back afterwards. This is the valid S3 run	Empty

Both soak tests were shortened (10 and 2 minutes instead of 30) to save time; they check for errors, not for slow leaks over half an hour.

5.3 API memory: closed

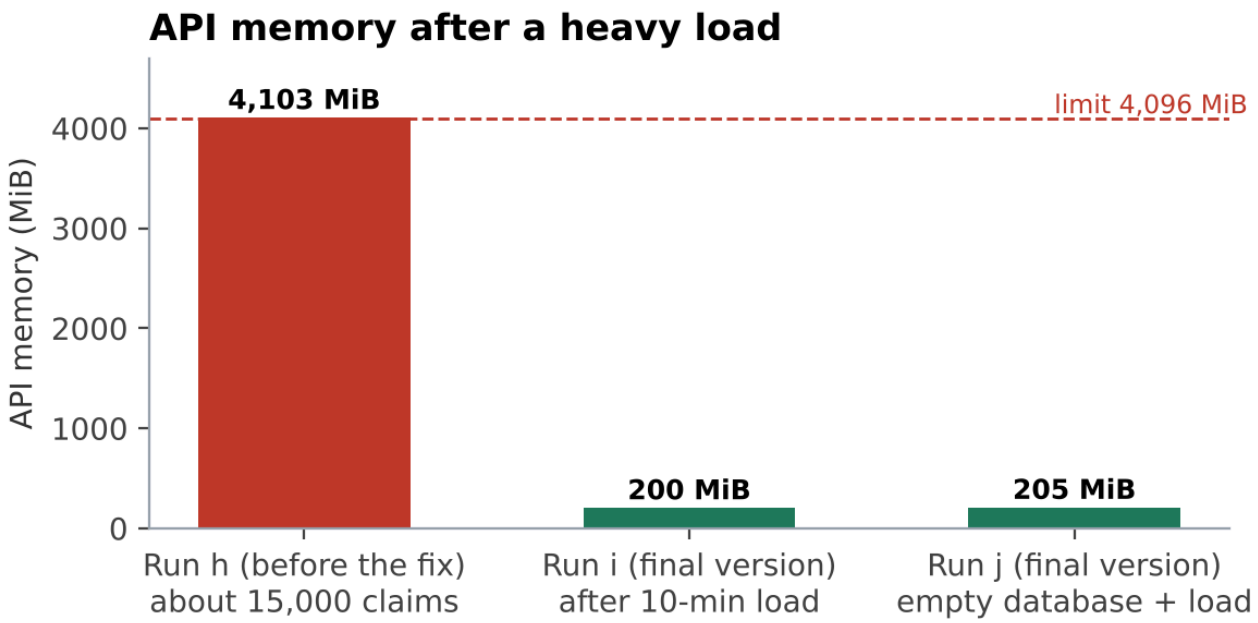


Figure 11. Memory used by the API after a heavy load: before the fix (run h) and with the final version (runs i and j). Dashed line: the 4 GiB limit.

Moment	API	Worker
Run h, peak, about 15,000 claims stored	**4,103 MiB (the limit is 4,096)**	2,544 MiB (limit 3,072)
Final version, right after the restart	**40 MiB**	969 MiB
Run i, after S2 (3,160 claims sent) and S7	**200 MiB**	982 MiB
Run j, empty database, after S2 and S7	**205 MiB**	441 MiB

> **What it means.** The cause found in the first analysis, loading every claim into memory for the dashboard and health pages, is removed. The API now stays near 200 MiB under the same kind of load, 5 % of its limit. The 4 GiB limit can stay as a safety margin, or be lowered to free memory for other services. Memory was read once after each run with `docker stats`, not sampled during it, so a short peak inside a run was not seen.

5.4 Pipeline and steady load

Batch	Run h (before)	Run i (final version, about 15,000 claims)
10 claims	1,943 a minute	2,314 a minute
50	2,932	2,942
200	3,789	3,149
500	4,140	3,112

Every stage is far above the target of 200 a minute. With large batches the final version is about 25 % slower than run h; the two runs used different amounts of stored data and different code, so the cause of this difference was not isolated. The steady load of 100 claims a minute for **10 minutes** (run i) had no failed batch in any minute: the typical batch took 0.26 s in every minute, and the slowest 5 % took 0.26 to 0.40 s, except 1.4 s in the first minute. A 2-minute steady load also ran on the empty database (run j); its per-minute results were not read for this edition.

5.5 Read pages and the number of stored claims

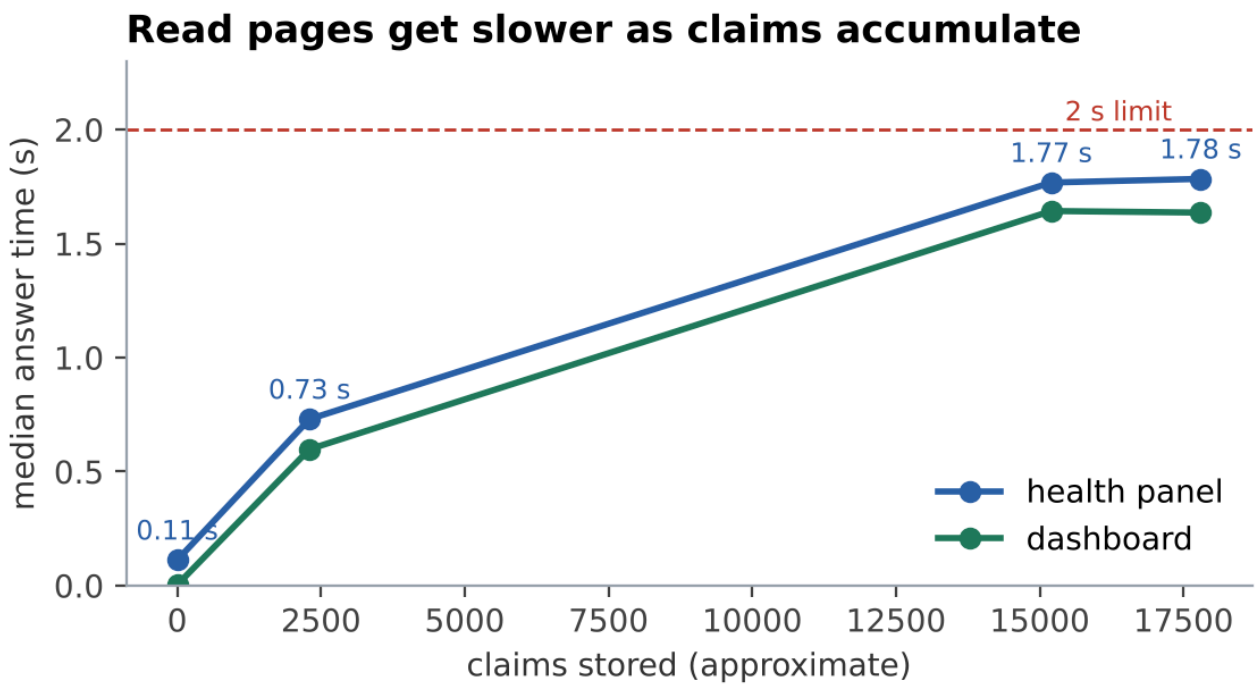


Figure 12. Median answer time of the health panel and the dashboard against the number of stored claims. The claim counts are approximate (see the note below); limit: 2 seconds.

Stored claims (approximate)	Dashboard	Health panel	Claims list
0 (emptied database, before the load)	**0.003 s**	**0.11 s**	0.002 s
About 2,300 (after the load of run j)	0.60 s	0.73 s	0.002 s
About 15,200 (run i, before the load)	**1.64 s**	**1.77 s**	0.002 s
About 17,800 (run i, after the load)	**1.64 s**	**1.78 s**	0.002 s

Run h started with about 8,500 claims and ended with about 15,000; run i started near 15,200 and the database held 18,325 claims at its end (counted when it was emptied). The counts at the measuring moments are estimated from the number of claims each run sent.

> **What it means.** This settles the question left open in the second edition: the 2-second read pages are caused by the amount of stored data, not by the load. Streaming fixed the memory but not the time, because every request still reads all the claims. With a real insurer's volume the page would be slower still. The fix is to keep the totals for the dashboard and the health panel in a summary that is refreshed every 10 to 30 seconds, and to read the summary on each request. This is a change to the project, not a setting; until then, keep the stored claims in the low thousands or accept about 2 s on these two pages.

5.6 Search by meaning, measured again

In the first report the search speed (2 to 4 ms, up to 1,819 answers a second) was measured while the same questions were asked again and again, so the answer cache served them. The new script asks **cold** questions (never asked before) separately from repeated ones. Run k showed why it needs the rate limit raised: 282 of its requests were refused with HTTP 429 and no speed was measured. Run l, with the limit at 1,000 a minute for the run only, is the valid result. The normal limit (30 a minute per person) was restored afterwards and checked.

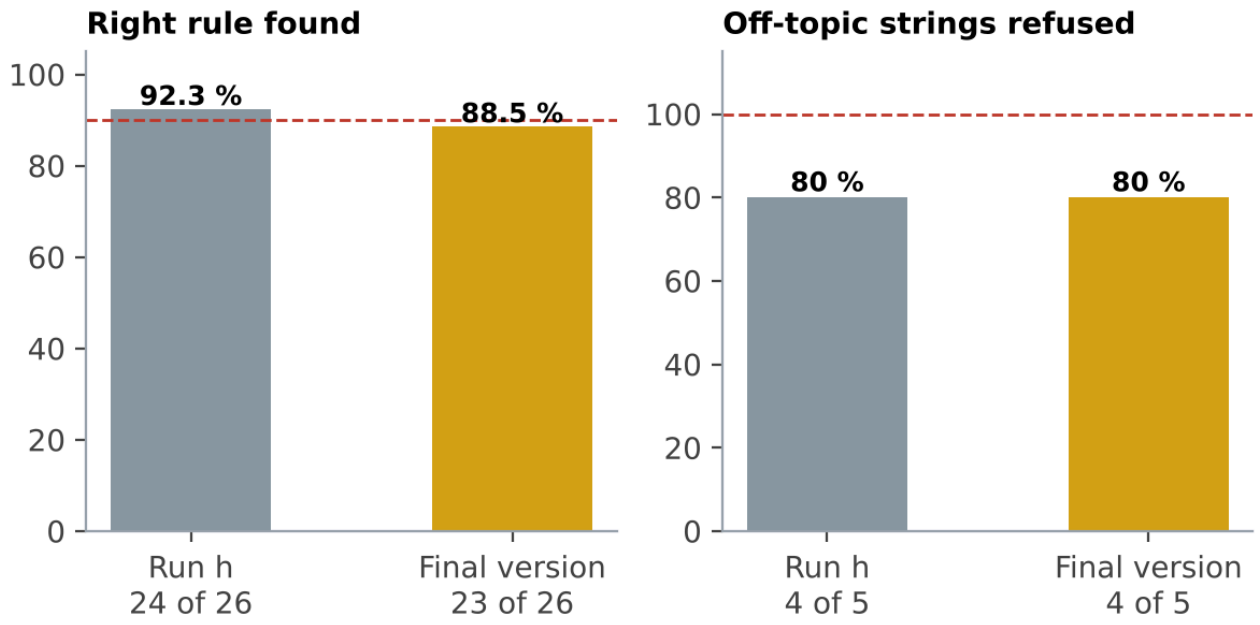


Figure 13. Quality of the search, run h against the final version. Dashed line: the 90 % pass line (left) and 100 % (right).

Measure	Run h	Final version (run l)
Right rule found, 26 questions	24 (92 %)	**23 (88.5 %), line 90 %**
Off-topic strings refused	4 of 5	4 of 5 (one in five still answered)
One question at a time (median / slowest 5 % / slowest)	3.8 s / 5.2 s / not recorded	1.1 s / 9.1 s / 60 s
Cold questions, 8 at once: 78 asked, 78 answered	Not measured	Median 15 s as the client saw it; 8.1 s on the server (slowest 5 %: 12.6 s); 0.56 questions a second
Requests refused by the rate limit	295 (test artefact)	0

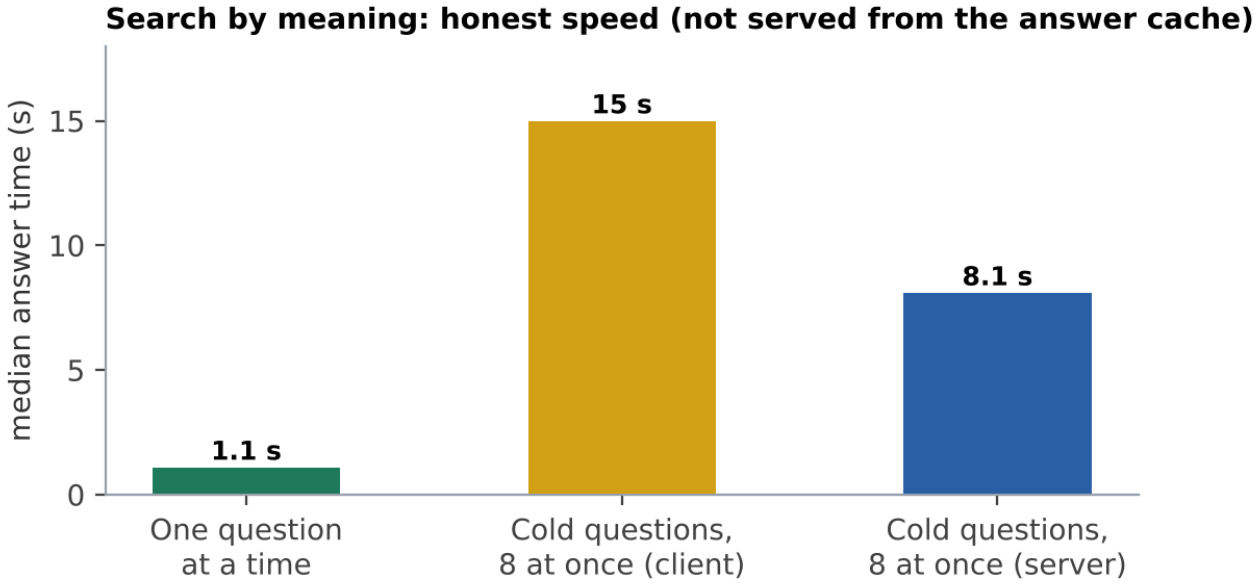


Figure 14. Median time for a question asked for the first time.

The three misses of run l. (1) "what happens if the invoice number is missing" returned an error: it was the only request of the first pass that did not return 200, and the slowest request of the run took 60

s, so it was probably a timeout while the models were loading after the API restart. This was not confirmed; if it were a one-off, the result would be 24 of 26 (92 %). (2) "claim line without a service date" was answered, but not with rule R001, the one the test expects. (3) "how many days after the service can a claim be submitted" was answered without rule R014 and the two policy tables.

> **What it means.** The search quality did not change with the update and sits one question under the line. The honest speed is about 1 s for a question asked alone and about 8 s on the server under 8 at once, almost all of it the explanation model on the GPU, so the "2 to 4 ms" belongs only to repeated questions. The figures for repeated (warm) questions in run l were not read for this edition; they are in `s03_search.json` in the results folder.

5.7 Integrity, and the "flow" probe explained

Check	Run i (about 18,000 claims)	Run j (empty database)
Health panel answers	**Yes**	**Yes**
Audit chain verifies	**Yes**	Not read for this edition
Ledger verifies	**Yes: HTTP 200 in 0.19 s; security log 86 checkpoints, 22,854 rows, 22,671 covered, 183 newer than the last checkpoint; no problems listed**	Not read for this edition
Probe `flow`	**Degraded**	**ok**
Probe `timestamp`	**Unknown**	**Unknown**
All other probes (anchor, ledger, signals, queue_store, broker, scheduler, audit_guard, outbox, model)	**ok**	**ok**
Script's "no failing probe" check	**False (because of flow)**	Not read

> **What it means.** The project's document 66 says the `flow` probe turns degraded when claims have been waiting for more than 30 minutes with nobody on shift. The test database held thousands of such claims and no staff account was signed in. The probe is "ok" on the emptied database, which fits this explanation. The probe's own detail text was not read, so this is a well-supported reading, not a proof. The `timestamp` probe is "unknown" because no external timestamp service is configured; it is not a failure. The HTTP 502 of run h's first S7 attempt did not repeat in runs i and j, and its likely cause, the API's memory limit, is closed (section 5.3).

5.8 Cleaning the test data

The database held **18,325 test claims**, 1,724 batch events, 400 notifications, 34 cache entries and 8 jobs. A compressed dump of all of it (348 MB, `~/mongo-test-data.gz`) was taken first. Then only these five collections were emptied, the Redis queue was restarted and all containers came back healthy. **The accounts (7), known clients (7), the configuration, the deals and the patient keys (50) were kept**, so nothing had to be set up again. The web console answered in 3.7 ms right afterwards. The benchmark accounts (CG-4002, CG-4003 and CG-1001) are still there and should be deleted once a real administrator exists.

5.9 Verdict of the follow-up

Item	Result
API memory	**Closed: 205 MiB (was 4,103 MiB)**
Failed soak batch of the first run	**Did not repeat in the 10- and 2-minute loads (the 30-minute load was not repeated)**
Dashboard and health panel speed	**Open: grows with stored claims; cached summaries needed**
Probe "flow"	**Explained (waiting test claims)**
Search speed in the first report	**Corrected (it was the answer cache)**
Search quality	**88.5 %, one question under the line; off-topic 4 of 5**
Phone photos, explanation speed, Ollama memory, quality of explanations	**Not re-tested in the follow-up**

6. Findings and recommendations

#	Finding	Status after the final version	Recommended action	Priority
1	API at its memory limit; HTTP 502 once (run h)	**Closed: 205 MiB after load**	Check `docker stats` weekly; the 4 GiB limit can be lowered	**Done**
2	Dashboard and health panel 1.6 to 1.8 s with 15,000 to 18,000 claims; 3 ms and 0.11 s when empty	**Open, cause confirmed**	Cache the dashboard and health totals for 10 to 30 s in the API; until then keep stored claims low or accept 2 s	**High at volume**
3	Photos read at 60 % (S4)	**Open**	Test `OCR_DPI` 200 and the project's prompt options; accept PDFs and scans and keep human review	**High**
4	Search quality 23 of 26 (88.5 %), off-topic 4 of 5	**Open**	Test `EMBEDDER_THRESHOLD` slightly higher (for example 0.50) against the 26 questions and the 5 strings; re-run the first question after a quiet start to see if the timeout repeats	**Medium**
5	Explanations 210 a minute (S1)	**Not re-run**	Enough for the 200 target, no more; raise `OLLAMA_NUM_PARALLEL` or return to vLLM on a larger card if the load grows	**Medium**
6	Ollama at 99 % of its 5 GiB limit (S0)	**Not re-checked**	Watch it; raise the limit if the container is ever killed (exit code 137)	**Medium**
7	One 99 GB volume, 75 % used; filled twice on 1 November	**Open**	Apply the split and routine of section 2.6; check the Proxmox host first	**Medium**
8	Cold search question takes about 1 s alone and about 8 s under 8 at once	**New, measured**	Budget for it; the answer cache only helps repeated questions	**Low**
9	Probe "timestamp" unknown	**Open**	Configure an external timestamp service, or accept it as a known limit	**Low**
10	Probe "flow" degraded	**Explained**	Confirm in the health panel's detail text; expect "ok" when no claims wait	**Low**
11	API restart interrupts for about 5 s (S5)	**Unchanged**	Update in quiet hours; clients retry refused requests	**Low**
12	Ollama image and vision model not pinned	**Unchanged**	Record the image digest and model hashes; convert the official weights before production (document 45)	**Low**

7. Problems met and how each was solved

These are recorded so the next installation, update or rebuild after a failure goes faster. The first part covers 1 November (the last five rows are new in this edition); the second part is from the first installation.

7.1 Since the first report

Problem	Cause	Solution
Build froze the server (memory 98 %, swap full); Ctrl+C did nothing	Building the new image with the AI containers running on 12 GiB	Restart from Proxmox; raise memory to 16 GiB; stop the model and search containers before a build
Disk 100 % full (first time)	About 20 GB of build cache left by the rebuild	`docker builder prune -af` after every rebuild
Vision model failed on every photo (HTTP 500); photo score unchanged	Not enough GPU memory beside vLLM; the reader silently falls back to Tesseract and only logs a warning	Serve both models through Ollama (section 2.5). Check the reader's warnings when a score does not change
Disk 100 % full (second time); Ollama container 30 GB	Each out-of-memory crash of the vision model wrote a core dump inside the container	Remove the container (`docker compose rm -sf ollama` , without `-v` , so the model volume stays); add `ulimits: {core: 0}` to it
"The database cannot be reached"; API 503; sign-in failed	MongoDB crashed when the disk filled and stayed stuck; the API never reconnected	`docker compose restart mongo` , then `up -d` for the others. Data was intact
Vision model on the CPU: 2 to 4 minutes per photo; 39 of 40 documents failed	Test client waited 60 seconds	Run the model on the GPU (about 2 s per photo); the test client now waits 15 minutes
Model-check showed 0 of 15 right after a restart	The model was still loading	Wait until the container is healthy (vLLM) or for the first request to load the model (Ollama), then run it again
`docker compose` without the local settings in a new terminal	The command line is not stored between sessions	Define the compose command in each new shell, with `compose.local.yml` listed last
File copy into the reader container refused: read-only	The reader's filesystem is read-only on purpose	Test it from the host through the container's network address
S1 could not find the model server	The script looks for the vLLM container	Pass the model address (`--model-base`) and set `VLLM_MODEL=gemma3:4b`
Pasted Python edits did not match	PuTTY removes leading spaces when pasting	Search for text without leading spaces; check the count before replacing
`curl https://localhost/ui/` returns 000	Caddy answers by name or server address, not "localhost"	Use the server's IP address or the domain
Wrong file names when copying the update (`Claim_Guard-main (3).zip`)	Spaces and brackets in the name	Copy with the exact name in quotes
Search speed test: 282 requests refused (HTTP 429)	The search rate limit (30 a minute per person) is below the test's rate	Add `ASK_PER_MINUTE=1000` to `.env` , restart the API, run S3, remove the line, restart again and check that `grep -c ASK_PER_MINUTE .env` prints 0
Database backup failed: "command listDatabases requires authentication"	MongoDB has a root login; the first command had no credentials	Run `mongodump` inside the container with the login it already holds in its environment (command in section 10.2)

Result folders not shown by `git status`	`.gitignore` ignores `outputs/`	`git add -f` with the exact folders, then check `git status` before committing
The updated zip contained no new code	The zip was the version already installed (only results folders differed)	Compare with `diff -rq` against the installed version before updating
Pasted multi-line Python lost its indentation	PuTTY removes leading spaces	Use one-line commands, or a here-document without indentation

7.2 From the first installation

Problem	Cause	Solution
`curl` not found	Minimal system	Install curl and other basic tools with apt
GPU not usable in Docker (BPF error)	Proxmox container blocks cgroup control	Run the NVIDIA toolkit in no-cgroups mode
GPU libraries missing in containers	Versioned library files absent	Copy the `.so.1` files to versioned names; use CDI device access
Model download refused (403)	Gemma is a gated model; token empty	Accept the licence on Hugging Face; set the token silently; revoke a token that had been exposed
vLLM would not start (missing CUDA 13 libraries; compile error; no KV-cache memory)	Libraries not in the image; `/tmp` without exec; too little GPU memory reserved	Mount `/opt/nvrtc13` and set `LD_LIBRARY_PATH`; `tmpfs /tmp:size=4g,exec`; `VLLM_GPU_UTIL=0.85` (these concern the former vLLM setup)
Browser error SSL protocol	No default certificate name for access by IP	`default_sni` in Caddy
"That request came from another site"	The API compares the Origin header with the Host header; the proxy sent the internal name	`header_up Host claimguard.kernelsi.com` in each proxy block
Search service would not start (offline error)	Model name not resolvable offline	Point `EMBED_MODEL` to the local model folder
Test scripts blocked by Cloudflare (error 1010)	Python user agent refused	Run the scripts against the server's IP address
Site returned 502; API exit code 137	Memory limit 768 MiB too low for thousands of claims	Raise the API and worker limits (now 4 GiB and 3 GiB)

8. Checklist before real use

1. **Create the real administrator** (`cg admin create-admin`) and then delete the benchmark accounts CG-4002, CG-4003 and CG-1001. The test claims are already removed (section 5.8).
2. **Decide about the dashboard.** Either add the cached summaries (section 5.5) before real volumes arrive, or accept about 2 s on the dashboard and health panel above roughly 15,000 stored claims.
3. **Apply the storage split and routine** of section 2.6, check the Proxmox host's free space, and set an alert at 80 % used.
4. **Protect the backups.** `backup-before-update*/` contains the encryption key; store it, with the `.env` copy, `compose.local.yml` and the 348 MB test-data dump, off the server and off the Downloads folder

(encrypted drive or password manager). Never commit them to GitHub. Delete the dump if the test data is not needed.

5. **Rotate the `OCR_TOKEN`** that appeared in a screenshot, delete the Hugging Face token that is no longer needed, keep SSH (port 22) closed to the internet and consider limiting ports 80 and 443 to Cloudflare's ranges.
6. **Delete temporary files** on the server: `~/bench.env`, `~/ph`, `~/rt.py`, `~/env.backup`, `~/compose.local.backup`, `~/results_h.tgz`, `~/results_ijkl.tgz`, `~/specs.txt`, the update zips and temporary folders. Delete the temporary clone on the PC.
7. **Pin the Ollama image** (record its digest) and the model files.
8. **Schedule a backup** of the database and run a restore test (command in section 10.2).
9. **Re-test photos** with a higher image resolution, and repeat the 30-minute steady load once on the final version.

9. What this test does not cover

- All claims and documents were **synthetic**. Real hospital paper, handwriting and Arabic documents are untested.
- Results are for **one server** and one user at a time on the console. Many reviewers working together were not simulated, and photos and explanations were not sent at the same time in a mixed-load test (they were tested one after the other, with a model swap between).
- The time to swap between the explanation model and the vision model was not measured on its own.
- Accuracy of the AI explanations (are they correct and useful to a reviewer?) was not scored, with vLLM or with Ollama. S1 measures speed only.
- The 30-minute steady load was not repeated on the final version (10 and 2 minutes were), and the speed of repeated (warm) search questions was not read in this edition.
- The search tests use 26 questions and 5 off-topic strings; a few questions change the percentage a lot (one question is 3.8 points).
- The storage figures marked "not recorded" (section 2.6) need the command of section 10.5.
- Failure of the whole machine, loss of the disk and restoration from backup were not tested.
- Security testing here is a view from the network, not a penetration test.

10. Appendix

10.1 Files in the repository (branch khalil_test)

File	Content
`docs/66_Server_Experiment_Results.md`	Results as written by the script (rebuild it for run `server_20261101h`)
`docs/figures/server/`	Charts of the run
`docs/reports/ClaimGuard_Server_Report_v3.pdf`	This report (third edition). The second edition stays in the same folder
`outputs/server/server_20261101i, j, k, l/`	Raw results of the follow-up runs (section 5.2)
`outputs/server/server_20261101h/`	Raw results (report.md, the experiment files and s06_exposure.json)
(not in Git) `deploy/rust/compose.local.yml`	The server's own settings; the project keeps it out of Git on purpose. Content in section 10.4

10.2 Commands used

Purpose	Command
The compose command of this server (define it in every new shell)	`DC="docker compose -f deploy/rust/compose.yml -f deploy/rust/compose.gpu.yml -f deploy/rust/compose.local.yml --env-file .env --profile documents --profile search"`
Download the explanation model into Ollama	`$DC --profile ollama-fetch run --rm ollama-fetch 'ollama serve >/tmp/o.log 2>&1 & sleep 8; ollama pull gemma3:4b; ollama list'`
Run all experiments (inside tmux, in the virtual environment)	`export VLLM_MODEL=gemma3:4b; python experiments/server/run_all.py --run server_20261101h --docs ~/docs_test --model-base http://<ollama address>:11434/v1 --soak-minutes 30 --yes-restart --compose "$DC"`
Run only the documents test	`python experiments/server/run_all.py --run server_20261101h --only s04 --docs ~/docs_test`
Integrity check on its own	`python experiments/server/s07_integrity.py --run server_20261101h`
Exposure test (from another machine)	`python experiments\server\s06_exposure.py --run server_20261101h --host claimguard.kernelsi.com`
Check the models answer	`$DC exec -T worker cg model-check | tail -2`
Free disk space after a build	`docker builder prune -af`
Raise the search limit for S3 only	`echo 'ASK_PER_MINUTE=1000' >> .env; $DC up -d api`; after the run: `sed -i '/^ASK_PER_MINUTE=/d' .env; $DC up -d api; grep -c ASK_PER_MINUTE .env` (must print 0)
Back up the database (login taken from the container itself)	`$DC exec -T mongo sh -c 'mongodump --archive --gzip --username "$MONGO_INITDB_ROOT_USERNAME" --password "$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase admin' > ~/mongo-test-data.gz`
Empty the test claims, keep accounts and settings	Stop api and worker; in `mongosh` on database `claimguard_queue`, run `deleteMany({})` on `claims`, `batch_events`, `notifications`, `cache` and `jobs`; restart Redis; `$DC up -d`
Restore a dump (only if needed)	`$DC exec -T mongo sh -c 'mongorestore --archive --gzip --username "$MONGO_INITDB_ROOT_USERNAME" --password "$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase admin' < ~/mongo-test-data.gz`
Run the follow-up experiments	`python experiments/server/run_all.py --run server_20261101i --only s02,s07 --soak-minutes 10` (with `CG_URL=https://<server address>` and the `CG_*` variables set)
Add result folders to Git	`git add -f outputs/server/server_20261101i ...` (the folder `outputs/` is ignored by default)
Listening ports on the server	`sudo ss -tlnp`

10.3 Glossary

Term	Meaning
Slowest 5 % (p95)	95 out of 100 requests were at least this fast
Median (p50)	The typical request: half were faster, half slower
OCR	Optical character recognition: reading text from an image
Vision model	An AI model that looks at an image and reads it; here the second reader for photos, next to Tesseract
vLLM	A program that runs an AI model on the GPU at high speed; it keeps most of the card reserved
Ollama	A program that runs AI models on the GPU and loads and unloads them as requests arrive, so several models can share one card
Core dump	A file a crashed program writes with a copy of its memory; it can be many gigabytes
TLS	The encryption behind HTTPS; versions 1.0 and 1.1 are obsolete
HSTS	A header that tells browsers to always use HTTPS
Soak test	A steady load over a long time to reveal slow leaks and rare failures
Exit code 137	The container was killed, usually for exceeding its memory limit
HTTP 429 / 502 / 503	Too many requests (rate limit) / the proxy could not reach the API / the service is unavailable

10.4 The server's own settings (`deploy/rust/compose.local.yml`, summarised)

Part	What it sets
api	Memory limit 4 GiB; explanation model address `http://ollama:11434`, model name `gemma3:4b`
worker	Explanation model address and name as for the api
reader	Vision model address `http://ollama:11434/v1`, model `richardyoung/unlimited-ocr`, page time limit 300 s, image resolution 130 dpi
ollama	Image `ollama/ollama:latest`; GPU access; models kept loaded for 30 minutes; one model loaded at a time; 2 requests at once; memory limit 5 GiB; no core dumps (`ulimits: core 0`); on the database and reader networks; model volume `ollama_data`
ollama-fetch	One-off container with internet access that downloads models into the volume (profile `ollama-fetch`)

10.5 Command that records the server's specifications and storage

Run on the server; it reads only and prints no passwords. It lists the items marked "not recorded" in sections 2.1 and 2.6, so the next edition can show them as numbers.

```
{ echo "== OS"; head -4 /etc/os-release; uname -r; echo "== CPU"; lscpu | grep -E 'Model name|^CPU\(s\)|L3'; echo "== RAM"; free -h; swapon --show; echo "== GPU"; nvidia-smi --query-gpu=name,memory.total,driver_version,power.limit --format=csv; echo "== DISK"; lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT; df -hT /; echo "== DOCKER"; docker info 2>/dev/null | grep -E 'Server Version|Storage Driver|Docker Root Dir'; docker system df; docker system df -v; echo "== MODELS"; docker exec claimguard-ollama-1 ollama list; echo "== FOLDERS"; du -sh ~/Claim_Guard-main ~/backup* ~/*.gz ~/*.zip 2>/dev/null; } > ~/specs.txt 2>&1
```